

Cross-Chain Attestation Portability

Stefan Stefanović, Ligate Labs

2026-05-26

Cross-Chain Attestation Portability

Unified Mechanism for IBC, Hyperlane, and Restaking-Style Bridging

Ligate Labs Research, Working Paper v0.1

Date: 2026-05-26

Status: v0.1 outline. Section structure and intent established; substantive content lands at v0.2. No formal claims yet. Authoring opens because all five upstream papers (native-delegation v0.2, per-schema-fees v0.2, cross-schema-composition v0.2, time-locked-attestations v0.2, native-da v0.2) shipped 2026-05-25 and each references this paper as the unified follow-up for its cross-chain extension. v0.2 substantive content depends on the Ligate Labs engineering decision on bridging target (IBC vs Hyperlane vs restaking-style).

Contact: hello@ligate.io

Version history: v0.1 (2026-05-26, outline).

Contents

Cross-Chain Attestation Portability	1
Unified Mechanism for IBC, Hyperlane, and Restaking-Style Bridging	1
Abstract	3
1. Introduction	3
1.1 Why one paper for five extensions	3
1.2 The chain-on-Celestia rollup-portability constraint	3
1.3 The IBC-vs-Hyperlane-vs-restaking choice	3
1.4 The central question	3
1.5 Approach in brief	3
1.6 Contributions	4
1.7 Scope and non-goals	4
1.8 Document structure	4
2. Background and Related Work	4
2.1 IBC (Inter-Blockchain Communication)	4
2.2 Hyperlane	4
2.3 Wormhole, Polymer, LayerZero	5
2.4 Restaking-style attestation portability (EigenLayer-adjacent)	5
2.5 Ethereum cross-validator portability (EIP-7002 / EIP-7251)	5
3. The Unified IBC Light-Client Proof Primitive	5
3.1 Proof structure	5
3.2 Freshness commitment	5
3.3 Revocation-status query	5
3.4 Cascade adaptation	6
4. Per-Extension Mechanisms	6
4.1 Cross-chain delegation (native-delegation v0.2 §10.2)	6
4.2 Cross-chain fee-market portability (per-schema-fees v0.2 §9.4)	6
4.3 Cross-chain composition (cross-schema-composition v0.2 §9.1)	6
4.4 Cross-chain time-locks (time-locked-attestations v0.2 §9.2)	6
4.5 Cross-chain DA-layer queries (native-da v0.2 §13.4)	6
5. Update-Latency vs Revocation-Latency Analysis	7
5.1 Bounded-stale window	7
5.2 Per-extension damage bounds	7
5.3 Mitigations	7
6. Cross-Chain Slashing-Event Validation	7
6.1 Slashing events reported to Ligate via IBC	7
6.2 Slashing events reported by Ligate to counterparties	7
6.3 Race conditions and double-slash prevention	8
7. Roadmap	8
7.1 v0: no cross-chain	8
7.2 v1.5: first extension shipped	8
7.3 v2: multiple extensions	8
7.4 v3+: full surface	8
8. Conclusion	8
References	9
Appendix A: Per-extension worked example	9
Appendix B: IBC packet format spec	9

Abstract

[v0.1: one paragraph stating the contribution. Five existing v0.2 papers each defer a cross-chain extension to a unified follow-up. This paper consolidates those five extensions (cross-chain delegation, cross-chain fee-market portability, cross-chain composition, cross-chain time-locks, cross-chain DA-layer queries) into one coherent design built on a single primitive: an IBC-style light-client proof of Ligate-side state that counterparty chains can verify locally. The paper specifies the proof primitive, the per-extension mechanism per upstream paper, the update-latency vs revocation-latency trade-off, and the cross-chain slashing-event validation. The framing throughout: one shared mechanism rather than five overlapping ad hoc designs.]

1. Introduction

1.1 Why one paper for five extensions

[v0.1: Each of the five upstream papers identifies a cross-chain extension as out of scope; each notes the same shape of mechanism (IBC light-client proof + freshness commitment + cascade adaptation). Specifying these in five separate papers would re-derive the same primitives five times. This paper specifies the shared mechanism once and applies it per extension in §5.]

1.2 The chain-on-Celestia rollup-portability constraint

[v0.1: Ligate Chain runs as a Sovereign SDK rollup on Celestia. IBC-aware Cosmos chains expect IBC-native light-client proofs; Sovereign SDK rollup state proofs are not native IBC primitives in most Cosmos chains today. The paper documents this engineering gap and specifies the bridge primitive that closes it; the actual chain-side implementation is engineering work tracked separately.]

1.3 The IBC-vs-Hyperlane-vs-restaking choice

[v0.1: Three target options. (a) Pure IBC: maximum interoperability with Cosmos ecosystem; requires Ligate light-client portability. (b) Hyperlane: interoperator-validator-based message passing; easier to deploy initially, weaker security than IBC. (c) Restaking-style (EigenLayer-adjacent): use Ethereum L1 as the trust root for cross-chain Ligate messages. v0.2 substantive content commits to one target; v0.1 stays target-agnostic and specifies the primitive in IBC’s vocabulary.]

1.4 The central question

[v0.1: Given an attestation on Ligate Chain, what is the minimum chain-side primitive that lets an IBC-connected counterparty chain (or a Hyperlane-bridged chain, or a restaking-style verifier) verify the attestation’s existence, current validity, and (when applicable) the cascade-status of its dependencies, without trusting an off-chain coordinator?]

1.5 Approach in brief

[v0.1: §3 surveys related work. §4 specifies the unified IBC light-client proof primitive. §5 walks the five upstream papers’ extensions and shows how each uses the primitive. §6 analyzes the update-latency vs revocation-latency trade-off per extension. §7 specifies cross-chain slashing-event validation. §8 lays out the roadmap (v0 none, v1.5 one extension, v2 multiple, v3+ full).]

1.6 Contributions

1. **Unified mechanism.** One primitive (IBC light-client proof + freshness commitment) handles all five upstream papers' cross-chain extensions.
2. **Per-extension mechanism specification.** Five subsections, one per upstream paper, showing how the unified primitive is consumed.
3. **Latency analysis.** Bounded damage per extension under realistic IBC round-trip latency.
4. **Cross-chain slashing-event validation.** PoUA reputation on Ligate accepting a slashing event reported via IBC; counterparty chains validating Ligate-side slashes against their own slashing modules.
5. **Roadmap.** Phased rollout from no cross-chain (v0) through full surface (v3+).

1.7 Scope and non-goals

In scope:

- Unified primitive specification (IBC light-client proof + freshness)
- Per-extension mechanism for each of the five upstream papers
- Update-latency vs revocation-latency trade-off
- Cross-chain slashing-event validation
- Roadmap aligned with chain-side engineering pace

Explicitly out of scope:

- Picking the engineering target (IBC vs Hyperlane vs restaking) — engineering decision
- Specifying the Sovereign SDK rollup-state-proof bridge (engineering gap, documented but not closed by this paper)
- EVM-compatible bridge wrappers (out of scope; chain primitives only)
- Cross-chain fiat / off-ramp mechanics
- Bridge-protocol endorsement (the paper is neutral on partner choice)

1.8 Document structure

[v0.1: §2 surveys IBC / Hyperlane / Wormhole / Polymer / restaking-style alternatives. §3 specifies the unified IBC light-client proof primitive. §4 walks the five per-extension mechanisms. §5 covers latency analysis. §6 covers cross-chain slashing. §7 lays out the roadmap. §8 concludes.]

2. Background and Related Work

[v0.1: Survey of bridge / portability designs in the broader ecosystem.]

2.1 IBC (Inter-Blockchain Communication)

[v0.1: Cosmos SDK's IBC. Light-client-based, trust-minimized, mature. Standard model for Cosmos-to-Cosmos chain communication. Limitation: requires light-client portability of the chain's state proofs, which Sovereign SDK rollups do not currently have as a native IBC primitive.]

2.2 Hyperlane

[v0.1: Interoperator-validator-based message passing. Easier to deploy than IBC; trust model is

“Hyperlane operator validator set is honest” rather than “the chain’s light client is correct.” Weaker security but easier interoperability across non-Cosmos chains.]

2.3 Wormhole, Polymer, LayerZero

[v0.1: Compare and contrast briefly. Trust models, latency profiles, ecosystem adoption.]

2.4 Restaking-style attestation portability (EigenLayer-adjacent)

[v0.1: Use Ethereum L1 (or another base layer) as the trust root for cross-chain Ligate messages. Validators stake on Ethereum; their stake is slashable for cross-chain message correctness. Different security model: trust the restaking-validator set rather than the chain’s light client.]

2.5 Ethereum cross-validator portability (EIP-7002 / EIP-7251)

[v0.1: Ethereum’s own approach to cross-validator portability via EIP-7002 (triggerable exits) and EIP-7251 (max effective balance). Reference for what the broader ecosystem considers state-of-the-art for cross-chain validator messaging.]

3. The Unified IBC Light-Client Proof Primitive

[v0.1: The single mechanism that all five upstream papers’ cross-chain extensions consume.]

3.1 Proof structure

[v0.1: An IBC packet carries: (a) the attestation object itself, (b) a Merkle proof of the attestation’s inclusion in Ligate’s state tree at some block height h , (c) the Ligate-side header at h signed by the attester set quorum, (d) a freshness commitment (the most recent finalized height known to the counterparty, used to bound staleness). The counterparty light client verifies each component locally.]

3.2 Freshness commitment

[v0.1: The freshness commitment is what makes the primitive bounded-stale. Without it, an attacker could replay an old IBC packet showing an attestation that has since been revoked. The freshness commitment names the most recent header the counterparty has seen; the proof is only valid if it points to a header $\geq$ that commitment. The counterparty advances its commitment as new headers come in.]

3.3 Revocation-status query

[v0.1: Beyond the attestation’s existence, the counterparty often needs the attestation’s *current* status (valid / revoked / cascade-invalidated). The IBC primitive supports a “current-state-of-attestation” query that returns a signed Ligate-side status response. Latency: one IBC round-trip per query.]

3.4 Cascade adaptation

[v0.1: When CSC v0.2 §5 cascade fires on Ligat, dependent attestations on counterparty chains do not auto-update; the counterparty must query (per §3.3) or wait for the next IBC update. Cascade-visibility-latency bound: one IBC round-trip plus header-finality time.]

4. Per-Extension Mechanisms

[v0.1: How each of the five upstream papers' cross-chain extensions uses the §3 primitive.]

4.1 Cross-chain delegation (native-delegation v0.2 §10.2)

[v0.1: A transaction on counterparty chain C wants to be authorized by a delegation grant on Ligat. The counterparty receives an IBC packet carrying (the grant, its inclusion proof at height h , header h signed). Verifies the proof; admits the transaction. Revocation: counterparty's view of the grant is bounded-stale by one IBC update. Trade-off documented in §5.]

4.2 Cross-chain fee-market portability (per-schema-fees v0.2 §9.4)

[v0.1: A schema registered on Ligat is also recognized on counterparty C for fee-market purposes. C needs the current base fee b_σ to price its attestations of the same schema. The IBC primitive carries (b_σ, h) ; C 's fee-market reads this on each update. Bounded-stale by IBC update latency; counterparty's fee may slightly diverge from Ligat's during the window.]

4.3 Cross-chain composition (cross-schema-composition v0.2 §9.1)

[v0.1: A CSC v0.2 §4.3 typed reference can target an attestation on a counterparty chain. The Ligat-side admission check uses §3.1 in reverse: the counterparty's attestation is presented to Ligat with a counterparty-light-client proof. The Ligat-side admission predicate verifies the cross-chain proof. Cascade semantics: if the cross-chain input is revoked, Ligat cascade fires after the next IBC update reveals the revocation.]

4.4 Cross-chain time-locks (time-locked-attestations v0.2 §9.2)

[v0.1: A TLA on Ligat whose reveal is gated by counterparty block height. The Ligat-side admission predicate for the reveal accepts an IBC light-client proof of the counterparty's current height h_C . If $h_C \geq$ the time-lock target, the reveal admits. Latency: target height must be far enough in the future that IBC update latency is absorbed by the target's margin.]

4.5 Cross-chain DA-layer queries (native-da v0.2 §13.4)

[v0.1: A counterparty chain wants to query an attestation by its native-DA chunk. The IBC primitive carries the DA chunk pointer; the counterparty's DA light client (or a wrapping bridge) fetches and verifies. Operational dependency: counterparty needs sampling access to Ligat's native DA. v0.2 substantive content specifies whether this requires light-client portability (likely yes) or a custom bridge protocol.]

5. Update-Latency vs Revocation-Latency Analysis

[v0.1: All five extensions inherit the same shape of trade-off: the counterparty’s view of Ligate state is bounded-stale by IBC update cadence. Different extensions have different damage profiles under staleness.]

5.1 Bounded-stale window

[v0.1: With IBC updates every 30 seconds (typical Cosmos IBC), the counterparty’s view of Ligate state lags by 30 seconds in expectation. Revocation events on Ligate are visible to the counterparty after the next update.]

5.2 Per-extension damage bounds

[v0.1: - Cross-chain delegation (§4.1): up to one IBC window of transactions could be admitted on a counterparty using a now-revoked grant. Damage: bounded by the grant’s transaction-rate cap $\times$ IBC window. - Cross-chain fee-market (§4.2): bounded fee divergence during the window; no security implications, just price-precision loss. - Cross-chain composition (§4.3): up to one window of dependent attestations could admit referencing a now-cascaded-invalidated input. Damage: bounded; cascade fires retroactively on update. - Cross-chain time-locks (§4.4): no damage; the reveal predicate uses the counterparty’s current height directly, which is naturally fresh on the counterparty side. - Cross-chain DA queries (§4.5): bounded DA-fetch latency; no security implications if counterparty has light-client access.]

5.3 Mitigations

[v0.1: Two paths to reduce damage. (a) Faster IBC update cadence (more frequent updates, higher gas cost for the relay). (b) Pre-commitment patterns (Ligate-side actions signal pending revocation N blocks in advance, giving the counterparty time to drain). Both are governance-tunable and v0.2 specifies recommended values.]

6. Cross-Chain Slashing-Event Validation

[v0.1: PoUA reputation on Ligate accepting a slashing event reported via IBC; counterparty chains validating Ligate-side slashes against their own slashing modules.]

6.1 Slashing events reported to Ligate via IBC

[v0.1: If a Ligate-attestor-set member misbehaves on a counterparty chain (e.g., signs a conflicting attestation on a remote chain), the counterparty reports the misbehavior to Ligate via IBC. Ligate validates the proof, applies the slash via the `disputes` module, and updates PoUA reputation. Mechanism reuses the §3 primitive in reverse.]

6.2 Slashing events reported by Ligate to counterparties

[v0.1: If a counterparty-side attestor (whose cross-chain authority depends on Ligate-side slashable stake) misbehaves on the counterparty, the counterparty’s slashing module slashes the counterparty-side balance. Ligate’s view of the attestor’s reputation updates after the next IBC update.]

6.3 Race conditions and double-slash prevention

[v0.1: If misbehavior is simultaneously reported on both chains, the slashing-module on each chain must coordinate to avoid double-slash beyond the §A.3 cap. v0.2 specifies the coordination protocol.]

7. Roadmap

[v0.1: Phased rollout aligned with chain-side engineering.]

7.1 v0: no cross-chain

[v0.1: Devnet operates without any cross-chain extension. All five upstream papers' extensions are out of scope at v0.]

7.2 v1.5: first extension shipped

[v0.1: Ship the §3 primitive plus one extension. Recommendation: cross-chain delegation (§4.1) ships first because it has the clearest product use case (Iris-relayed agents acting across Cosmos chains).]

7.3 v2: multiple extensions

[v0.1: Add §4.2 (cross-chain fee-market) and §4.3 (cross-chain composition). Engineering work compounds; mainnet hardening accumulates.]

7.4 v3+: full surface

[v0.1: §4.4 (cross-chain time-locks) and §4.5 (cross-chain DA queries) ship as the cross-chain workload matures. Cross-chain slashing-event validation (§6) is co-shipped with the first extension that requires cross-chain attester coordination.]

8. Conclusion

[v0.1: Two paragraphs. (1) Cross-chain attestation portability is one shared mechanism, not five separate designs. The unified IBC light-client proof primitive plus freshness commitment handles all five upstream papers' extensions. Bounded staleness is the shared property; per-extension damage analysis follows from IBC update cadence. (2) The engineering choice (IBC vs Hyperlane vs restaking) is downstream of this paper; the paper specifies the primitive in IBC vocabulary but the mechanism generalizes. Roadmap is phased; v3+ ships the full surface once the chain-side engineering supports it.]

References

[**v0.1:** References to fill in at v0.2. Anchors:]

1. PoUA paper (this repo, papers/poua/), arXiv:2605.25844.
 2. Native Delegation paper (this repo, papers/native-delegation/) §10.2.
 3. Per-Schema Fees paper (this repo, papers/per-schema-fees/) §9.4.
 4. Cross-Schema Composition paper (this repo, papers/cross-schema-composition/) §9.1.
 5. Time-Locked Attestations paper (this repo, papers/time-locked-attestations/) §9.2.
 6. Native DA Layer paper (this repo, papers/native-da/) §13.4.
 7. Cosmos IBC specification.
 8. Hyperlane technical documentation.
 9. EIP-7002 (Ethereum triggerable exits) specification.
 10. EIP-7251 (Ethereum max effective balance) specification.
 11. EigenLayer restaking specification.
-

Appendix A: Per-extension worked example

[**v0.1:** At v0.2: a worked example for the v1.5-first-extension recommendation (cross-chain delegation). Trace an Iris-relayed agent transaction on a Cosmos counterparty using a Ligate-side delegation grant: chain transitions on both sides, IBC packet contents, light-client verification steps, latency profile.]

Appendix B: IBC packet format spec

[**v0.1:** At v0.2: byte-level specification of the IBC packet format Ligate emits and consumes for each of the five extensions. Field layouts, signing, integrity checks.]